Hands On Inkubasi

Let's Play Git & Github

Daftar Isi

1. Git
2. GitHub
3. GitHub Container Registry (GHCR)
4. GitHub Actions

1. Git

> 💡 **Apa itu Git?**
> Git adalah tool *version control* yang berjalan di komputermu. Git men-tracking setiap perubahan file supaya kamu bisa membuat "checkpoint" atau "snapshot" yang biasa kita sebut *commit*. Ketika kumpulan commit tersebut sudah membentuk sesuatu yang layak untuk dirilis, kamu bisa menandainya sebagai sebuah *version.* Git pada dasarnya tidak butuh internet untuk bekerja.

1.1 Konfigurasi Awal

Sebelum mulai, kenalkan dirimu ke Git. Informasi ini akan tercatat di setiap commit yang kamu buat dan tersimpan di file konfigurasi Git di komputermu.

`git config`

Perintah untuk membaca dan menulis konfigurasi Git, mulai dari identitas pengguna hingga perilaku Git secara umum.

Flag	Keterangan
`--system`	Konfigurasi berlaku untuk semua user di komputer
`--global`	Konfigurasi berlaku hanya untuk user saat ini
`--local`	Konfigurasi berlaku hanya untuk repo saat ini
`--list`	Tampilkan semua konfigurasi yang aktif

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@kamu.com"
git config --list
```

> 📌 **3 Level Konfigurasi Git:**
>
> | Level | Flag | Linux / Mac | Windows |
> |---|---|---|---|
> | System | `--system` | `/etc/gitconfig` | `C:\Program Files\Git\etc\gitconfig` |
> | Global | `--global` | `~/.gitconfig` | `C:\Users\USERNAME\.gitconfig` |
> | Local | `--local` | `.git/config` (dalam repo) | `.git/config` (dalam repo) |

Prioritas: `local` > `global` > `system`. Jika terdapat ketiganya, maka konfigurasi local yang akan digunakan.

Contoh:

```
mkdir test
cd test
git init
git config --local user.name "Test 123"
git config list
cd ..
git config list
```

1.2 Inisialisasi Repository

`git init`

Membuat repository Git baru di folder saat ini. Perintah ini membuat folder tersembunyi `.git` yang berisi seluruh data version control.

Flag	Keterangan
`-b <nama>`	Tentukan nama branch utama (default: `master` atau `main`)

```
mkdir poros
cd poros
git init -b inkubasi
```

1.3 Staging & Commit

> 💡 Git menggunakan dua tahap sebelum perubahan tersimpan, yaitu:

> 1. Staging, memilih file yang akan disimpan/ditrack
> 2. Commit, menyimpan checkpoint-nya

`git status`

Menampilkan kondisi working directory saat ini. Informasi yang ditampilkan adalah nama file yang berubah, file yang sudah di-stage, dan file yang belum ter-track.

Flag	Keterangan
`-s`	Tampilkan output dalam format ringkas (short)

```
git status
git status -s
```

`git diff`

Menampilkan detail perubahan baris per baris yang belum atau sudah di-stage.

Flag	Keterangan
`--staged`	Tampilkan perubahan yang sudah ada di staging area

```
git diff
git diff --staged
```

`git add`

Memindahkan perubahan dari working directory ke staging area. Perintah ini digunakan untuk memilih perubahan mana yang akan masuk ke commit berikutnya.

Contoh penggunaan:

```
git add peserta.txt
# atau
git add .
```

`git commit`

Menyimpan snapshot dari semua perubahan yang ada di staging area sebagai sebuah commit baru.

Flag	Keterangan
`-m "<pesan>"`	Tulis pesan commit langsung dari terminal
`-am "<pesan>"`	Stage semua file terlacak sekaligus commit

Flag	Keterangan
`--amend`	Ubah commit terakhir (pesan atau isinya)

```
git commit -m "feat: tambah fitur login"
git commit -am "fix: perbaiki bug validasi form"
git commit --amend -m "feat: tambah halaman login"
```

> 📌 **Contoh Konvensi Pesan Commit (Conventional Commits):**
>
> - `feat:` — fitur baru
> - `fix:` — perbaikan bug
> - `docs:` — perubahan dokumentasi
> - `refactor:` — refaktor kode

`git log`

Menampilkan riwayat commit dari yang paling baru ke paling lama.

Flag	Keterangan
`--oneline`	Tampilkan setiap commit dalam satu baris
`--graph`	Tampilkan struktur branch secara visual
`--all`	Tampilkan semua branch, bukan hanya yang aktif
`-n <angka>`	Batasi jumlah commit yang ditampilkan

```
git log
```

`git show`

Menampilkan detail lengkap dari sebuah commit, termasuk pesan commit, author, dan perubahan yang dilakukan.

```
git show
git show abc1234
git show v1.0.0
```

1.4 Branching

> 💡 **Branch** adalah suatu jalur modifikasi file yang saling terpisah dan terisolasi. Setiap branch dibuat dari sebuah **base commit** yang sama, namun perubahan di dalamnya dapat dilakukan secara bebas tanpa mempengaruhi branch lain. Nantinya, antar branch dapat digabungkan kembali melalui proses yang disebut **merge**.

`git branch`

Membuat, menampilkan, atau menghapus branch.

Flag	Keterangan
`-a`	Tampilkan semua branch termasuk remote
`-d <nama>`	Hapus branch yang sudah di-merge
`-D <nama>`	Paksa hapus branch meskipun belum di-merge

```
git branch
git branch -a
git branch -d nama-fitur
```

`git checkout`

Berpindah ke branch lain atau membuat branch baru sekaligus pindah ke sana.

Flag	Keterangan
`-b <nama>`	Buat branch baru sekaligus pindah ke sana (`checkout`)

```
git checkout -b nama-fitur
git checkout nama-fitur
```

1.5 Merging & Rebasing

`git merge`

Menggabungkan perubahan dari satu branch ke branch yang sedang aktif.

Flag	Keterangan
`--squash`	Gabungkan semua commit dari branch menjadi satu sebelum merge
`--abort`	Batalkan proses merge yang sedang conflict

```
git checkout main
git merge nama-fitur
```

`git rebase`

Memindahkan rangkaian commit ke atas base branch yang baru, sehingga history menjadi lebih linear.

```
git checkout nama-fitur
git rebase main
```

1.7 Undo & Reset

`git restore`

Membatalkan perubahan pada file, bisa di working directory maupun di staging area.

Flag	Keterangan
`--staged <file>`	Keluarkan file dari staging (batalkan `git add`)

```
git restore nama-file.txt
git restore --staged nama-file.txt
```

`git reset`

Memindahkan pointer HEAD ke commit tertentu, dengan pilihan bagaimana nasib perubahan setelahnya.

Flag	Keterangan
`--soft HEAD~1`	Hapus commit terakhir, perubahan tetap di staging
`--mixed HEAD~1`	Hapus commit terakhir, perubahan kembali ke working directory
`--hard HEAD~1`	Hapus commit terakhir beserta seluruh perubahannya

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Studi Kasus:
Kamu terlanjur commit terlalu cepat dan ingin menggabungkannya dengan commit berikutnya.

```
git reset --soft HEAD~1
git add file-tambahan.txt
git commit -m "feat: fitur baru + tambahan"
```

1.8 Tagging

`git tag`

Menandai sebuah commit dengan label yang mudah diingat, biasanya digunakan untuk menandai versi rilis.

Flag	Keterangan
`-m "<pesan>"`	Tambahkan pesan pada tag dengan anotasi
`-d <tag>`	Hapus tag

```
git tag v1.0.0 -m "Release versi 1.0.0"
git tag
```

---

2. GitHub

> 💡 **Apa itu GitHub?**
> GitHub adalah platform hosting untuk repository Git. Di sinilah kamu menyimpan kode secara online, berkolaborasi dengan tim, dan memanfaatkan fitur seperti Pull Request, Issues, dan Actions. GitHub adalah produk terpisah dari Git. GitHub dibangun di atas Git, bukan bagian darinya.

2.1 Fork

> 💡 **Apa itu Fork?**
> Fork adalah fitur milik **platform hosting** (GitHub, GitLab, Bitbucket, dll.), **bukan** fitur bawaan Git. Git sendiri tidak punya perintah `git fork`.
>
> Secara teknis, fork bekerja seperti `clone` yang dilakukan di sisi server GitHub, dengan tambahan:
>
> - Menyimpan relasi antara repo hasil fork dan repo aslinya
> - Memudahkan sinkronisasi perubahan dari repo asal (upstream)
> - Memfasilitasi alur Pull Request kembali ke repo aslinya
>
> Fork sangat umum digunakan dalam open-source: kamu bebas bereksperimen di fork milikmu tanpa mempengaruhi repository asli.

Cara Fork Repository:

1. Buka repository `https://github.com/mijonOfTheMoon/hello-poros`
2. Klik tombol **Fork** di pojok kanan atas
3. Pilih akun tujuan, lalu klik **Create fork**
4. Sekarang kamu punya salinan repo tersebut di akunmu sendiri

---

2.2 Clone Repository

`**git clone**`

Menyalin repository remote ke komputer lokal secara lengkap, termasuk seluruh history commit dan branch-nya. Perintah ini otomatis men-setup remote `origin` yang mengarah ke URL sumber.

Flag	Keterangan
`--depth <n>`	Hanya ambil n commit terakhir
`-b <branch>`	Clone branch tertentu langsung

Clone fork yang sudah dibuat di langkah sebelumnya:

```
git clone https://github.com/USERNAME/hello-poros.git
cd hello-poros
```

2.3 Remote

`git remote`

Mengelola daftar remote, yaitu alamat repository yang ada di server (seperti GitHub).

Flag	Keterangan
`add <nama> <url>`	Tambahkan remote baru
`-v`	Tampilkan daftar remote beserta URL-nya
`remove <nama>`	Hapus remote

```
git remote -v
# origin  https://github.com/USERNAME/hello-poros.git (fetch)
# origin  https://github.com/USERNAME/hello-poros.git (push)
```

2.4 Push & Pull

`git push`

Mengirim commit dari repo lokal ke remote repository.

Flag	Keterangan
`-u origin <branch>`	Push sekaligus set tracking branch (cukup sekali)
`--tags`	Push semua tag sekaligus
`--force`	Paksa push meski ada konflik history (hati-hati!)

```
git push -u origin main
git push
```

```
git push --tags
```

`git pull`

Mengambil perubahan terbaru dari remote dan langsung menggabungkannya ke branch lokal.

Flag	Keterangan
`--rebase`	Gunakan rebase alih-alih merge saat pull
`<remote> <branch>`	Pull dari remote dan branch tertentu

```
git pull
```

`git fetch`

Mengambil informasi terbaru dari remote tanpa langsung menggabungkannya ke branch lokal.

Flag	Keterangan
`--all`	Fetch dari semua remote sekaligus
`--prune`	Hapus referensi branch remote yang sudah tidak ada

```
git fetch origin
git fetch --all
```

2.5 Pull Request (PR)

> 💡 **Pull Request** adalah cara mengusulkan perubahan kode ke branch atau repo lain. PR memungkinkan proses review, diskusi, dan approval sebelum kode digabung.

Cara Membuat Pull Request:

1. Push branch kamu ke fork: `git push origin nama-fitur`
2. Buka repository di GitHub
3. Klik **Compare & pull request** (muncul otomatis) atau masuk ke tab **Pull requests** → **New pull request**
4. Pilih **base repository** (repo tujuan) dan **base branch** (misal: `main`)
5. Pilih **head repository** (fork kamu) dan **compare branch** (nama-fitur)
6. Isi judul dan deskripsi, tambahkan reviewer jika perlu
7. Klik **Create pull request**

2.6 Secrets & Environment Variables

> 💡 **Secrets** adalah variabel sensitif (API key, password) yang disimpan terenkripsi di GitHub dan bisa diakses oleh GitHub Actions.

Cara Menambahkan Secret:

1. Masuk ke **Settings** → **Secrets and variables** → **Actions**
2. Klik **New repository secret**
3. Isi **Name** (misal: `DOCKER_PASSWORD`) dan nilainya
4. Klik **Add secret**

Akses di workflow:

```
env:
  MY_SECRET: ${{ secrets.DOCKER_PASSWORD }}
```

---

3. GitHub Container Registry (GHCR)

> 💡 **GHCR** (`ghcr.io`) adalah registry Docker milik GitHub. Kamu bisa menyimpan dan mendistribusikan Docker image langsung dari ekosistem GitHub, terintegrasi penuh dengan repository dan Actions.

3.1 Buat Dockerfile

Repository `hello-poros` yang sudah di-fork sudah berisi file `index.html`. Buat `Dockerfile` di root folder repository:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

3.2 Build Image Lokal

```
docker build -t ghcr.io/USERNAME/hello-poros:latest .
docker images
docker run -p 8080:80 ghcr.io/USERNAME/hello-poros:latest
```

3.3 Login ke GHCR

Login menggunakan Personal Access Token (PAT). Buat PAT di: GitHub → Settings → Developer settings → Personal access tokens. Scope yang diperlukan: write:packages, read:packages

```
echo "TOKEN_KAMU" | docker login ghcr.io -u USERNAME --password-stdin
```

3.4 Push & Pull Image

```
docker tag nama-app:latest ghcr.io/USERNAME/hello-poros:v1.0.0
docker push ghcr.io/USERNAME/hello-poros:v1.0.0
```

```
docker pull ghcr.io/USERNAME/hello-poros:v1.0.0
```

3.5 Menggunakan Image dari GHCR di docker-compose

```yaml
name: hello-poros
services:
  app:
    container_name: hello-poros
    image: ghcr.io/USERNAME/hello-poros:latest
    ports:
      - "8080:80"
```

```
docker compose up -d
docker compose ps
```

---

4. GitHub Actions

> 💡 **GitHub Actions** adalah platform CI/CD bawaan GitHub. Kamu bisa mengotomasi proses build, test, dan deploy menggunakan file YAML yang disimpan langsung di dalam repository.

4.1 Konsep Dasar

Istilah	Penjelasan
Workflow	File YAML yang mendefinisikan seluruh proses otomasi
Event	Pemicu workflow (push, pull_request, schedule, dll.)
Job	Sekumpulan steps yang berjalan di satu runner
Step	Satu perintah atau action dalam sebuah job
Action	Unit kerja yang bisa dipakai ulang (dari Marketplace atau kustom)
Runner	Server yang menjalankan job (ubuntu, windows, macos)

4.2 Self-Hosted Runner (Ubuntu)

> 💡 **Self-hosted runner** adalah server milikmu sendiri yang didaftarkan untuk menjalankan job GitHub Actions. Berguna jika kamu ingin deploy langsung ke server, membutuhkan akses ke resource lokal, atau ingin menghemat menit Actions gratis.

Download:

```
mkdir actions-runner && cd actions-runner

curl -o actions-runner-linux-x64-2.332.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.332.0/actions-runner-
linux-x64-2.332.0.tar.gz
```

```
tar xzf ./actions-runner-linux-x64-2.332.0.tar.gz
```

Configure and Install as a Service:

```
./config.sh --url https://github.com/USERNAME/nama-repo --token TOKEN_DARI_GITHUB

sudo ./svc.sh install

sudo ./svc.sh start

sudo ./svc.sh enable
```

Menggunakan di Workflow:

```yaml
jobs:
  deploy:
    runs-on: self-hosted
```

4.3 Event Pemicu (Triggers)

Event menentukan **kapan** workflow akan dijalankan. Satu workflow bisa memiliki lebih dari satu trigger.

```yaml
on:
  push:
    branches: [ "main", "develop" ]
    paths:
      - 'src/**'           # Hanya trigger jika file di src/ berubah

  pull_request:
    branches: [ "main" ]

  schedule:
    - cron: '0 8 * * 1'   # Setiap Senin jam 08:00 UTC

  workflow_dispatch:       # Trigger manual dari GitHub UI
    inputs:
      environment:
        description: 'Deploy target'
        required: true
        default: 'staging'
        type: choice
        options: [staging, production]
```

4.4 Contoh Workflow: Build, Push, dan Deploy

Berikut contoh workflow lengkap yang mem-build Docker image, push ke GHCR, lalu deploy di self-hosted runner.

```yaml
name: Build & Push Docker Image

on:
  push:
    branches: [ "main" ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  build-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    outputs:
      image-tag: ${{ steps.meta.outputs.tags }}  # Expose tag agar bisa dipakai job lain

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4  # Clone repo ke runner

      - name: Login ke GitHub Container Registry
        uses: docker/login-action@v3  # Autentikasi ke ghcr.io
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Ekstrak metadata (tags, labels)
        id: meta
        uses: docker/metadata-action@v5  # Generate tag dan label otomatis (sudah lowercase)
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: latest

      - name: Build dan Push image
```

```
      uses: docker/build-push-action@v5  # Build dari dockerfile lalu push ke registry
      with:
        context: .
        push: true
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}

  deploy:
    runs-on: self-hosted
    needs: build-push  # Tunggu job build-push selesai
    steps:
      - name: Deployment
        run: |  # Pull dan jalankan image yang baru saja di-push
          docker pull ${{ needs.build-push.outputs.image-tag }}
          docker run -d ${{ needs.build-push.outputs.image-tag }}
```

4.5 Conditional Steps

Gunakan `if` untuk menjalankan step tertentu hanya jika kondisi terpenuhi, misalnya hanya di branch tertentu atau hanya jika job sebelumnya berhasil.

```
steps:
  - name: Hanya jalankan di main branch
    if: github.ref == 'refs/heads/main'
    run: echo "Ini production!"

  - name: Hanya jika job sebelumnya berhasil
    if: success()
    run: echo "Sukses!"

  - name: Selalu jalankan (termasuk jika ada error)
    if: always()
    run: echo "Cleanup..."
```

📚 Referensi

Topik	Link
Git Documentation	https://git-scm.com/doc
GitHub Docs	https://docs.github.com
GitHub Actions Marketplace	https://github.com/marketplace?type=actions

Topik	Link
GitHub Container Registry	https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry
Conventional Commits	https://www.conventionalcommits.org
Docker Documentation	https://docs.docker.com